M2DS - École Polytechnique

Cloud Data and Graph Mining

Project - Knowledge Extraction through Graph Mining

By:

Jérémie Formey de Saint Louvent

Ralph Mouawad

To:

Prof. Nicolas Travers

Contents

1 Introduction **3**

2 Node Similarity **3**

2.1 Similarity Metrics 3

2.2 Queries, Results, and Interpretation 5

2.2.1 User similarity with Jaccard 5

2.2.2 User similarity with Overlap 6

2.2.3 User similarity with cosine on ratings 7

2.2.4 Movie similarity with Jaccard 9

2.2.5 Movie similarity with Pearson correlation 10

2.2.6 Movie similarity using embeddings (FastRP + KNN) 11

2.2.7 Highly similar users with cosine similarity and demographic comparison 12

2.2.8 Highly similar users with Jaccard similarity and demographic comparison 13

3 Community Detection and Centrality Analysis **15**

3.1 Methods 15

3.2 Queries, Results, and Interpretation 17

3.2.1 Louvain on the user-user Jaccard graph 17

3.2.2 Top movie genres per community 19

3.2.3 Betweenness centrality on the bipartite user-movie graph 20

3.2.4 Top users by betweenness 21

3.2.5 PageRank on the user-user Jaccard graph 22

4 Link Prediction **23**

4.1 Method 23

4.2 Cypher Query 24

4.3 Result 25
4.4 Interpretation 25

5 Conclusion **25**
5.1 Limitations 26

List of Figures

1 Introduction

In this project, we model the MovieLens dataset as a bipartite graph

$$G = (U \cup M, E),$$

where U is the set of users, M is the set of movies, and $E \subseteq U \times M$ represents the interactions between users and movies (i.e. ratings). An edge $(u, m) \in E$ indicates that user $u \in U$ has rated movie $m \in M$. This structure allows us to capture user-item relationships in a natural and interpretable way.

From this bipartite graph, we construct a mono-partite user-user graph

$$G_U = (U, E_U),$$

where edges $(u, v) \in E_U$ connect users based on similarity measures derived from their shared interactions with movies. The edge weights typically reflect similarity scores such as Jaccard or cosine similarity, enabling the analysis of relationships between users.

The objective of this work is to study various graph-based approaches to extract meaningful insights from the data. In particular, we explore node similarity measures, community detection algorithms, and centrality metrics to understand the structure of the network. Additionally, we investigate link prediction techniques to infer potential future interactions, such as predicting how a user might rate unseen movies.

All experiments are implemented using Neo4j, leveraging its Graph Data Science (GDS) library. Queries are written in Cypher, allowing efficient graph projection, algorithm execution, and result analysis directly within the database.

2 Node Similarity

2.1 Similarity Metrics

In this project, multiple similarity measures were used to compare users and movies in the MovieLens graph. Each metric captures a different notion of proximity, either through direct neighborhood overlap, rating behavior, or learned structural representations.

Jaccard Similarity For two nodes x and y with neighborhoods $N(x)$ and $N(y)$,

$$J(x, y) = \frac{|N(x) \cap N(y)|}{|N(x) \cup N(y)|}.$$

This metric measures the proportion of shared neighbors relative to the total number of distinct neighbors. In our graph, it is used to compare users through common watched movies, and movies through common audiences.

Overlap Similarity

$$O(x, y) = \frac{|N(x) \cap N(y)|}{\min(|N(x)|, |N(y)|)}.$$

This metric highlights containment. It becomes equal to 1 when the smaller neighborhood is entirely included in the larger one. It is useful for detecting subset behavior between users.

Cosine Similarity Given two vectors $\mathbf{r}_x$ and $\mathbf{r}_y$,

$$\cos(x, y) = \frac{\mathbf{r}_x \cdot \mathbf{r}_y}{\|\mathbf{r}_x\| \, \|\mathbf{r}_y\|}.$$

This metric measures angular similarity between vectors. In this project, it is used to compare users based on their ratings over shared movies.

Pearson Similarity For two items i and j rated by the same users,

$$\rho(i, j) = \frac{\sum_{u \in U_{ij}} (r_{u,i} - \bar{r}_i)(r_{u,j} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{u,i} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_{ij}} (r_{u,j} - \bar{r}_j)^2}}.$$

Pearson similarity measures how strongly two rating patterns vary together after centering by their mean. It is particularly useful for comparing movies independently of absolute rating scale.

FastRP Embeddings FastRP is a graph embedding method that maps each node to a low-dimensional vector:

$$\mathbf{z}_x \in \mathbb{R}^d.$$

These embeddings summarize the structural position of nodes in the graph by incorporating information from their neighborhoods. Unlike local similarity metrics such as Jaccard, FastRP can capture higher-order graph structure.

K-Nearest Neighbors (KNN) Once embeddings are computed, KNN is used to retrieve the most similar nodes in the embedding space. For a node x, the method finds the

set

$$\mathcal{N}_k(x) = \{y_1, \ldots, y_k\}$$

of the k closest nodes according to a similarity function, typically cosine similarity. In our case, KNN is applied to movie embeddings in order to identify structurally similar movies.

2.2 Queries, Results, and Interpretation

2.2.1 User similarity with Jaccard

This query measures how similar two users are based on the set of movies they both watched.

```
CALL gds.nodeSimilarity.filtered.stream(
  'movielens_bipartite',
  {
    sourceNodeFilter: 'User',
    targetNodeFilter: 'User',
    similarityMetric: 'JACCARD'
  }
)
YIELD node1, node2, similarity
RETURN
  gds.util.asNode(node1).id AS user1,
  gds.util.asNode(node2).id AS user2,
  similarity
ORDER BY similarity DESC
LIMIT 10;
```

User 1	User 2	Jaccard Similarity
408	898	0.8387
898	408	0.8387
328	788	0.6730
788	328	0.6730
489	587	0.6299
587	489	0.6299
600	826	0.5455
826	600	0.5455
451	489	0.5333
489	451	0.5333

Table 1: Top user pairs by Jaccard similarity.

The highest Jaccard score is obtained by users 408 and 898, with a similarity of 0.8387, which indicates a very strong overlap in the movies they watched. The results also show symmetric duplicates such as $(408, 898)$ and $(898, 408)$, which is expected since similarity is undirected. Overall, this metric identifies users with highly similar viewing histories. These results can be used for neighbor-based recommendation, where movies watched by one user but not yet seen by the other can be recommended. They can also help detect groups of users with similar consumption patterns.

2.2.2 User similarity with Overlap

This query evaluates whether one user's watched set is largely contained within another user's watched set.

```
CALL gds.nodeSimilarity.stream('movielens_bipartite', {
  similarityMetric: 'OVERLAP'
})
YIELD node1, node2, similarity
WITH gds.util.asNode(node1) AS u1, gds.util.asNode(node2) AS u2, similarity
WHERE u1:User AND u2:User AND id(u1) < id(u2)
MATCH (u1)-[:RATED]->(m1:Movie)
WITH u1, u2, similarity, count(DISTINCT m1) AS watched1
MATCH (u2)-[:RATED]->(m2:Movie)
WITH u1, u2, similarity, watched1, count(DISTINCT m2) AS watched2
RETURN
```

```
  u1.id AS user1,
  watched1,
  u2.id AS user2,
  watched2,
  similarity
ORDER BY similarity DESC, abs(watched1 - watched2) DESC
LIMIT 10;
```

User 1	Watched 1	User 2	Watched 2	Overlap Similarity
655	685	866	20	1.0000
111	24	655	685	1.0000
131	30	655	685	1.0000
13	636	729	21	1.0000
13	636	163	23	1.0000
13	636	258	23	1.0000
13	636	547	23	1.0000
13	636	604	27	1.0000
13	636	191	27	1.0000
13	636	366	33	1.0000

Table 2: Top user pairs by overlap similarity.

All top returned pairs have an overlap similarity equal to 1, which means that the smaller watched set is entirely included in the larger one. For instance, user 866 watched only 20 movies, all of which were also watched by user 655. This does not necessarily mean the two users are globally similar, but rather that one user's history is a subset of the other's. This is useful for identifying dominant users, reference profiles, or cases where a small-profile user can inherit recommendations from a much richer profile. It is also helpful in understanding nested viewing behavior within the graph. We can see that user 13 has a lot of similarities with other users which indicate that he is a user of interest: many of other users watched histories are subsets of user 13's one.

2.2.3 User similarity with cosine on ratings

This query compares users based on the ratings they assigned to the movies they both rated, not only based on the watched movies.

```
MATCH (u1:User)-[r1:RATED]->(m:Movie)<-[r2:RATED]-(u2:User)
```

```
WHERE u1.id < u2.id
WITH u1, u2, m, r1.rating AS rating1, r2.rating AS rating2
ORDER BY m.id
WITH
  u1,
  u2,
  collect(rating1) AS ratings1,
  collect(rating2) AS ratings2,
  count(m) AS sharedMovies
WHERE sharedMovies >= 5
RETURN
  u1.id AS user1,
  u2.id AS user2,
  sharedMovies,
  gds.similarity.cosine(ratings1, ratings2) AS cosineSimilarity
ORDER BY cosineSimilarity DESC, sharedMovies DESC
LIMIT 10;
```

User 1	User 2	Shared Movies	Cosine Similarity
358	711	8	1.0000
563	932	7	1.0000
75	412	7	1.0000
158	415	7	1.0000
120	647	7	1.0000
404	659	7	1.0000
542	584	7	1.0000
640	777	7	1.0000
118	476	7	1.0000
47	514	7	1.0000

Table 3: Top user pairs by cosine similarity on ratings.

All returned pairs have a cosine similarity of 1, meaning that their rating vectors are perfectly aligned over the movies they rated in common. This indicates extremely similar rating behavior, although the number of shared movies remains relatively small, between 7 and 8. Therefore, the similarity is strong but based on limited overlap. These results are valuable for collaborative filtering, since such users are very good candidates for predicting

each other's missing ratings. At the same time, the low number of shared movies suggests that this measure should be interpreted carefully, as perfect similarity on a small sample may be less robust than moderate similarity over a much larger set of common movies.

2.2.4 Movie similarity with Jaccard

This query finds pairs of movies that tend to be watched by the same users.

```
CALL gds.nodeSimilarity.stream('movielens_bipartite', {
  similarityMetric: 'JACCARD'
})
YIELD node1, node2, similarity
WITH gds.util.asNode(node1) AS m1, gds.util.asNode(node2) AS m2, similarity
WHERE m1:Movie AND m2:Movie AND id(m1) < id(m2)
RETURN
  m1.id AS movie1,
  m1.title AS title1,
  m2.id AS movie2,
  m2.title AS title2,
  similarity
ORDER BY similarity DESC
LIMIT 10;
```

Movie 1	Title 1	Movie 2	Title 2	Jaccard Similarity
1348	Every Other Weekend (1990)	1366	JLG/JLG - autoportrait de décembre (1994)	1.0000
1348	Every Other Weekend (1990)	1340	Crude Oasis, The (1995)	1.0000
1348	Every Other Weekend (1990)	1343	Lotto Land (1995)	1.0000
1348	Every Other Weekend (1990)	1373	Good Morning (1971)	1.0000
439	Amityville: A New Generation (1993)	437	Amityville 1992: It's About Time (1992)	1.0000
1348	Every Other Weekend (1990)	1329	Low Life, The (1994)	1.0000

Table 4: Top movie pairs by Jaccard similarity.

The returned movie pairs all have a Jaccard similarity of 1, which means they were watched by exactly the same users. This often happens for niche or rarely watched movies, where a very small audience watched the same set of titles. One interesting example is the pair of *Amityville* movies, which likely share both theme and audience. These results can be used to identify substitute or companion movies for recommendation. However, a Jaccard score of 1 should not automatically be interpreted as strong global similarity, since

it may be driven by very low support. In practice, this metric is especially useful when combined with the number of common viewers.

2.2.5 Movie similarity with Pearson correlation

This query compares movies based on the ratings they received from the same users.

```
MATCH (u:User)-[r1:RATED]->(m1:Movie)
MATCH (u)-[r2:RATED]->(m2:Movie)
WHERE m1.id < m2.id
WITH
  m1,
  m2,
  collect(r1.rating) AS ratings1,
  collect(r2.rating) AS ratings2,
  count(u) AS sharedUsers
WHERE sharedUsers >= 5
RETURN
  m1.id AS movie1,
  m1.title AS title1,
  m2.id AS movie2,
  m2.title AS title2,
  sharedUsers,
  gds.similarity.pearson(ratings1, ratings2) AS pearsonSimilarity
ORDER BY pearsonSimilarity DESC, sharedUsers DESC
LIMIT 10;
```

Movie 1	Title 1	Movie 2	Title 2	Shared Users	Pearson Similarity
111	Truth About Cats & Dogs, The (1996)	938	Smile Like Yours, A (1997)	6	1.0000
50	Star Wars (1977)	766	Man of the Year (1995)	7	1.0000
729	Nell (1994)	1124	Farewell to Arms, A (1932)	6	0.95
969	Winnie the Pooh and the Blustery Day (1968)	1147	My Family (1995)	6	0.95
242	Kolya (1996)	1171	Wild Reeds (1994)	6	0.88
77	Firm, The (1993)	1285	Princess Caraboo (1994)	6	0.88

Table 5: Top movie pairs by Pearson similarity.

All top movie pairs have a high Pearson similarity close to 1, which means the users who rated both movies gave them perfectly linearly correlated ratings. This reveals highly

consistent rating patterns across shared viewers. Unlike Jaccard similarity, which measures common audience, Pearson focuses on common appreciation. Therefore, two movies may have very different audiences overall but still exhibit strong rating correlation among users who watched both. This is particularly useful for item-based recommendation, since it helps suggest movies that are not only co-watched, but also similarly evaluated. As with previous results, the support remains limited to 6 or 7 shared users, so the strongest conclusions should be drawn carefully.

2.2.6 Movie similarity using embeddings (FastRP + KNN)

This query uses graph embeddings (FastRP) to project nodes into a latent vector space, then applies KNN to find structurally similar movies.

```
CALL gds.knn.stream('movielens_bipartite', {
  nodeProperties: ['embedding'],
  topK: 10,
  similarityCutoff: 0.7
})
...
```

Movie 1	Title 1	Movie 2	Title 2	Similarity
1320	Homage (1995)	1325	August (1996)	1.0000
1320	Homage (1995)	1349	Mille bolle blu (1993)	1.0000
1320	Homage (1995)	1364	Bird of Prey (1996)	1.0000
1320	Homage (1995)	1366	JLG/JLG - autoportrait...	1.0000
1320	Homage (1995)	1363	Leopard Son, The (1996)	1.0000
1320	Homage (1995)	1341	Hedd Wyn (1992)	1.0000
1320	Homage (1995)	1343	Lotto Land (1995)	1.0000

Table 6: Top movie similarities using FastRP embeddings and KNN.

The results show multiple pairs of movies with a similarity score equal to 1. This indicates that these movies are embedded at extremely close positions in the learned latent space. Such behavior is expected when dealing with sparse or low-degree nodes, where movies share very similar structural roles in the graph (e.g., being watched by the same small group of users).

Unlike Jaccard or Pearson similarity, embedding-based similarity captures higher-order relationships beyond direct co-occurrence. Therefore, these results suggest that the identified movies belong to tightly connected substructures or niche clusters in the graph.

From an application perspective, this approach is particularly useful for recommendation systems, as it can identify structurally similar items even when direct overlap is limited. However, the presence of many perfect similarities also highlights a limitation: embeddings may collapse nodes with very similar local neighborhoods, reducing discrimination between them. This suggests that combining embedding-based similarity with other metrics (e.g., Jaccard or Pearson) could lead to more robust recommendations.

2.2.7 Highly similar users with cosine similarity and demographic comparison

This query first identifies user pairs with very high cosine similarity in their rating behavior, then compares their demographic attributes.

```
MATCH (u1:User)-[r1:RATED]->(m:Movie)<-[r2:RATED]-(u2:User)
WHERE u1.id < u2.id
WITH
  u1,
  u2,
  collect(r1.rating) AS ratings1,
  collect(r2.rating) AS ratings2,
  count(m) AS sharedMovies
WHERE sharedMovies >= 5
WITH
  u1,
  u2,
  sharedMovies,
  gds.similarity.cosine(ratings1, ratings2) AS sim
WHERE sim IS NOT NULL AND sim > 0.9
WITH u1, u2, sim
ORDER BY sim DESC
LIMIT 3
RETURN
  u1.id AS user1,
  u2.id AS user2,
  sim AS similarity,
  u1.age AS age1,
  u2.age AS age2,
  abs(u1.age - u2.age) AS age_gap,
```

```
u1.gender AS gender1,
u2.gender AS gender2,
CASE WHEN u1.gender = u2.gender THEN 'same' ELSE 'different' END AS same_gender,
u1.occupation AS occupation1,
u2.occupation AS occupation2,
CASE WHEN u1.occupation = u2.occupation THEN 'same' ELSE 'different' END AS same_occupati
```

User 1	User 2	Sim.	Age 1	Age 2	Gap	G1	G2	Same G.	Same Occ.
191	259	1.0000	33	21	12	M	M	same	different
131	240	1.0000	59	23	36	F	F	same	different
118	137	1.0000	21	50	29	M	M	same	different

Table 7: Top highly similar users by cosine similarity with demographic comparison.

The three returned pairs have a cosine similarity of 1, meaning that their ratings are perfectly aligned on the movies they both evaluated. A first observation is that all pairs share the same gender, while none of them share the same occupation. The age gaps are also relatively large in two cases, reaching 29 and 36 years. This suggests that highly similar rating behavior does not necessarily imply similar demographic profiles, except perhaps for gender in these top examples. In other words, users can express very similar preferences even when they belong to different age groups or professional categories.

These results are useful because they show that behavioral similarity extracted from the graph may go beyond standard demographic segmentation. In recommendation systems, this means that relying only on age or occupation would miss valuable neighbors. Instead, graph-based preference similarity appears more informative for identifying useful recommendation pairs.

2.2.8 Highly similar users with Jaccard similarity and demographic comparison

This query identifies users with strong overlap in watched movies, then examines whether these users also share demographic characteristics.

```
MATCH (u1:User)-[:RATED]->(m:Movie)<-[:RATED]-(u2:User)
WHERE u1.id < u2.id
WITH u1, u2, count(DISTINCT m) AS inter
MATCH (u1)-[:RATED]->(m1:Movie)
```

```
WITH u1, u2, inter, count(DISTINCT m1) AS deg1
MATCH (u2)-[:RATED]->(m2:Movie)
WITH u1, u2, inter, deg1, count(DISTINCT m2) AS deg2
WITH u1, u2, inter * 1.0 / (deg1 + deg2 - inter) AS jaccard
WHERE jaccard > 0.5
WITH u1, u2, jaccard
ORDER BY jaccard DESC
LIMIT 3
RETURN
  u1.id AS user1,
  u2.id AS user2,
  jaccard,
  u1.age AS age1,
  u2.age AS age2,
  abs(u1.age - u2.age) AS age_gap,
  u1.gender AS gender1,
  u2.gender AS gender2,
  CASE WHEN u1.gender = u2.gender THEN 'same' ELSE 'different' END AS same_gender,
  u1.occupation AS occupation1,
  u2.occupation AS occupation2,
  CASE WHEN u1.occupation = u2.occupation THEN 'same' ELSE 'different' END AS same_occupati
```

User 1	User 2	Jaccard	Age 1	Age 2	Gap	G1	G2	Same G.	Same Occ.
408	898	0.8387	23	23	0	M	M	same	different
328	788	0.6730	51	51	0	M	M	same	same
489	587	0.6299	55	26	29	M	M	same	same

Table 8: Top highly similar users by Jaccard similarity with demographic comparison.

For the top Jaccard pairs, all users again share the same gender, and two out of the three pairs also share the same occupation. In addition, two pairs have exactly the same age, which is a much stronger demographic alignment than what was observed with cosine similarity. This suggests that overlap in watched movies may be more associated with similar user profiles than rating-based similarity. Users who consume very similar sets of movies may belong to comparable demographic groups, whereas users with similar ratings may simply have similar taste, regardless of their background.

This distinction is important for interpretation. Jaccard captures common exposure to content, while cosine captures similarity in evaluation. Therefore, Jaccard-based neighbors may be more useful for audience profiling and segmentation, whereas cosine-based neighbors may be more relevant for personalized recommendation. Together, these two results show that different similarity metrics uncover different types of relationships in the graph.

The demographic comparison confirms that behavioral similarity and demographic similarity are related, but not equivalent. In the cosine-based analysis, similar users did not share occupation and sometimes had large age differences, which shows that rating behavior can transcend demographic boundaries. In contrast, the Jaccard-based analysis revealed stronger demographic alignment, especially in age and occupation. This suggests that the set of movies a user watches may reflect demographic characteristics more directly than the exact ratings they assign. From an application viewpoint, these results support combining both demographic and graph-based features when building recommendation or segmentation models.

Across these first similarity experiments, the highest scores are often equal or very close to 1, both for users and movies. This indicates the existence of highly concentrated local structures in the graph, such as identical viewing sets, subset relations, or perfectly aligned ratings. These findings are useful for collaborative filtering, audience segmentation, item-item recommendation, and the identification of tightly related user or movie neighborhoods. At the same time, many of the strongest similarities rely on relatively small supports, so in practice these metrics should ideally be analyzed together with the number of shared movies or shared users.

3 Community Detection and Centrality Analysis

3.1 Methods

Weighted Jaccard Graph A user-user graph is first built from the bipartite user-movie graph. The weight between two users u and v is their Jaccard similarity:

$$w(u, v) = J(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|},$$

where $N(u)$ is the set of movies watched by user u. This produces a weighted user similarity graph.

Louvain Community Detection Louvain aims to partition the graph into communities by maximizing modularity:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j),$$

where A_{ij} is the edge weight between nodes i and j, k_i is the weighted degree of node i, m is the total graph weight, and $\delta(c_i, c_j) = 1$ if i and j belong to the same community and 0 otherwise. A higher modularity indicates a stronger community structure.

Genre Profile per Community Once user communities are detected, each community is profiled by averaging the binary genre indicators of the movies watched by its members. For a community C and genre g:

$$\text{score}(C, g) = \frac{1}{|E_C|} \sum_{(u,m) \in E_C} \mathbf{1}_g(m),$$

where E_C is the set of watched user-movie pairs involving users in community C.

Betweenness Centrality Betweenness centrality measures how often a node lies on shortest paths:

$$BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$

where σ_{st} is the number of shortest paths from s to t, and $\sigma_{st}(v)$ is the number of those paths passing through v. Nodes with high betweenness act as bridges.

PageRank PageRank measures node importance by recursively aggregating importance from neighbors:

$$PR(v) = \frac{1-d}{n} + d \sum_{u \in \mathcal{N}^-(v)} \frac{w_{uv}}{\sum_z w_{uz}} PR(u),$$

where d is the damping factor, n the number of nodes, and w_{uv} the edge weight. In the user-user graph, this identifies influential users connected to other influential users.

3.2 Queries, Results, and Interpretation

3.2.1 Louvain on the user-user Jaccard graph

This query projects a weighted user-user graph using Jaccard similarity, then applies Louvain to detect communities.

```
CALL gds.graph.project.cypher(
  'usersJaccard',
  'MATCH (u:User) RETURN id(u) AS id',
  '
  MATCH (u1:User)-[:RATED]->(m:Movie)<-[:RATED]-(u2:User)
  WHERE u1.id < u2.id
  WITH u1, u2, count(DISTINCT m) AS inter
  MATCH (u1)-[:RATED]->(m1:Movie)
  WITH u1, u2, inter, count(DISTINCT m1) AS deg1
  MATCH (u2)-[:RATED]->(m2:Movie)
  WITH u1, u2, inter, deg1, count(DISTINCT m2) AS deg2
  WITH u1, u2, inter * 1.0 / (deg1 + deg2 - inter) AS jaccard
  WHERE jaccard > 0.1
  RETURN id(u1) AS source, id(u2) AS target, jaccard AS weight
  '
);
CALL gds.louvain.write('usersJaccard', {
  relationshipWeightProperty: 'weight',
  writeProperty: 'communityId'
})
YIELD communityCount, modularity;
```

Community Count	Modularity
3	0.3249

Table 9: Global Louvain result on the weighted user-user graph.

The Louvain algorithm detects 3 communities with a modularity of 0.3249. This indicates a meaningful but moderate community structure: users are not randomly connected, yet the graph is not split into extremely isolated groups either. This is expected in a

recommendation setting, where many users share some common popular movies while still forming preference-based clusters.

Community demographics The following query summarizes the detected communities.

```
MATCH (u:User)
WHERE u.communityId IS NOT NULL
RETURN
  u.communityId AS community,
  count(*) AS size,
  round(avg(u.age), 2) AS avg_age,
  count(CASE WHEN u.gender = 'M' THEN 1 END) AS nb_males,
  count(CASE WHEN u.gender = 'F' THEN 1 END) AS nb_females,
  collect(DISTINCT u.occupation)[0..8] AS occupations
ORDER BY size DESC
LIMIT 10;
```

Community	Size	Avg. Age	Males	Females
846	490	33.26	365	125
938	452	34.92	304	148
148	1	32.00	1	0

Table 10: Demographic summary of the detected communities.

Two large communities dominate the graph: communities 846 and 938, with 490 and 452 users respectively. Their average ages are close, around 33–35 years, and both are male-dominated. The third community contains only one user, which can be interpreted as an outlier or isolated profile. This suggests that most users belong to two major viewing groups, while a very small number of users remain weakly connected to the rest of the graph.

Sample users from communities To better understand the communities, a small sample of users can be inspected.

```
MATCH (u:User)
WHERE u.communityId IS NOT NULL
```

```
WITH u.communityId AS community, collect(u)[0..6] AS users
UNWIND users AS u
RETURN
  community,
  u.id AS user,
  u.age AS age,
  u.gender AS gender,
  u.occupation AS occupation
ORDER BY community, user;
```

Community	User	Age	Gender	Occupation
148	219	32	M	programmer
846	22	25	M	writer
846	115	31	M	engineer
846	186	39	F	executive
846	244	28	M	technician

Table 11: Sample users from detected communities.

The sample confirms that the large communities are heterogeneous in occupation, which means the grouping is not driven only by demographics. Instead, the communities likely reflect shared movie consumption patterns. This is useful for audience segmentation: each community can be treated as a group with relatively coherent preferences for recommendation or market profiling.

3.2.2 Top movie genres per community

This query computes, for each community, the genres most represented among the watched movies.

```
MATCH (u:User)-[:RATED]->(m:Movie)
WHERE u.communityId IS NOT NULL
...
RETURN community, topGenres
ORDER BY community;
```

Community	Top 5 Genres
148	Drama (0.5357), Comedy (0.2857), Action (0.2500), Thriller (0.2143), Romance (0.1786)
846	Drama (0.3906), Comedy (0.3071), Action (0.2574), Thriller (0.2039), Romance (0.1921)
938	Drama (0.4318), Thriller (0.2774), Comedy (0.2634), Action (0.2497), Romance (0.2047)

Table 12: Top genre profile for each user community.

All three communities are dominated by *Drama*, which appears as the top genre in every case. However, some differences still emerge. Community 846 is relatively more comedy-oriented, while community 938 places *Thriller* above *Comedy*. Even the singleton community 148 follows a similar pattern, though it is not statistically reliable because it contains only one user. These results show that the communities share a broad preference structure but differ in secondary tastes. This can be used to tailor recommendation strategies at the community level: for example, drama-heavy content may be recommended globally, while thriller-oriented content may be more suitable for community 938.

3.2.3 Betweenness centrality on the bipartite user-movie graph

This query identifies the nodes that most frequently lie on shortest paths in the user-movie graph.

```
CALL gds.betweenness.stream('movielens_bipartite')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS n, score
RETURN
  labels(n) AS node_type,
  CASE
    WHEN n:User THEN n.id
    WHEN n:Movie THEN n.title
  END AS node,
  round(score, 2) AS betweenness
ORDER BY betweenness DESC
LIMIT 20;
```

Node Type	Node	Betweenness
User	405	249918.61
User	655	207763.05
User	181	140107.95
User	13	114291.13
User	234	67023.23

Table 13: Top 5 nodes by betweenness centrality.

The top betweenness scores are all associated with users, not movies. This means that certain users act as major bridges between parts of the graph, connecting otherwise distant users through shared movie interactions. Such users likely have broad and diverse viewing histories, making them structurally central. These nodes are important because they can strongly influence the connectivity of the network and may serve as anchor profiles in recommendation or diffusion analyses.

Interpretation with movie bridges Although the top 5 nodes are all users, the broader ranking also contains major movies such as *Scream (1996)*, *Contact (1997)*, and *The English Patient (1996)*. This suggests that popular or cross-audience movies also act as bridges, linking different user groups. Hence, betweenness captures not just popularity, but the ability of a node to connect multiple parts of the graph.

3.2.4 Top users by betweenness

This query focuses only on the most central users in the bipartite graph.

```
CALL gds.betweenness.stream('movielens_bipartite')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS n, score
WHERE n:User
RETURN
  n.id AS user,
  n.age AS age,
  n.gender AS gender,
  n.occupation AS occupation,
  round(score, 2) AS betweenness
ORDER BY betweenness DESC
LIMIT 20;
```

Since the previous top-5 ranking already contains only users, the same leading profiles appear here. The most central users are those with highly connecting viewing behavior, and they may be useful as representative or influential users in the network. In practice, these users can be studied more closely to understand how communities are linked and how information or recommendations may flow across the graph.

3.2.5 PageRank on the user-user Jaccard graph

This query computes PageRank on the weighted user-user similarity graph.

```
CALL gds.pageRank.stream('usersJaccard', {
  relationshipWeightProperty: 'weight'
})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS u, score
RETURN
  u.id AS user,
  round(score, 6) AS pagerank,
  u.age AS age,
  u.gender AS gender,
  u.occupation AS occupation,
  u.communityId AS community
ORDER BY pagerank DESC
LIMIT 20;
```

User	PageRank	Age	Gender	Occupation	Community
943	21.8209	22	M	student	846
938	16.0959	38	F	technician	938
939	15.3669	26	F	student	938
942	13.9068	48	F	librarian	846
940	13.8142	32	M	administrator	846

Table 14: Top 5 users by PageRank in the weighted user-user graph.

Unlike betweeness, which identifies bridge nodes, PageRank highlights users connected to other important users. The top-ranked users belong mainly to the two large communities 846 and 938, confirming that these communities dominate the similarity graph.

Interestingly, the top users are not all of the same demographic type, which suggests that structural importance is more related to graph position than to age, gender, or occupation alone. These high-PageRank users can be interpreted as influential or representative users inside their communities, making them valuable for community-level recommendation or influence analysis.

The community detection results reveal that the user-user similarity graph is organized around two main communities and one isolated outlier. The genre profiles show that all communities strongly favor drama, while secondary preferences such as comedy or thriller vary slightly across groups. Centrality analysis complements this picture: betweenness identifies bridge users who connect different parts of the graph, whereas PageRank highlights influential users embedded in the dense core of the main communities. Together, these results provide a richer understanding of the network structure and can support recommendation, segmentation, and user profiling tasks.

4 Link Prediction

In this part, we address the problem of predicting the rating that a user would assign to a movie not yet considered during inference. More precisely, for a user u and a movie m, the goal is to estimate

$$\hat{r}_{u,m}.$$

We use a user-based collaborative filtering approach, where the prediction is computed from ratings given by users with similar rating behavior.

4.1 Method

For a target user u and target movie m, we first identify neighboring users v who: (i) have rated movie m, and (ii) have sufficient overlap with u on other commonly rated movies. The similarity between u and v is measured using cosine similarity on their rating vectors:

$$\mathrm{sim}(u,v) = \frac{\sum_{i \in I_{uv}} r_{u,i} r_{v,i}}{\sqrt{\sum_{i \in I_{uv}} r_{u,i}^2} \sqrt{\sum_{i \in I_{uv}} r_{v,i}^2}}.$$

The predicted rating is then computed as a weighted average:

$$\hat{r}_{u,m} = \frac{\sum_{v \in \mathcal{N}(u)} \mathrm{sim}(u,v)\, r_{v,m}}{\sum_{v \in \mathcal{N}(u)} |\mathrm{sim}(u,v)|}.$$

4.2 Cypher Query

The following query predicts the rating of user 123 for the movie *Fargo (1996)* while excluding the target movie from the similarity computation.

```
MATCH (u:User {id: 123})-[rTarget:RATED]->(m:Movie {id: 100})
WITH u, m, rTarget.rating AS actualRating

MATCH (u)-[r1:RATED]->(m_shared:Movie)<-[r2:RATED]-(v:User)
WHERE v <> u AND m_shared <> m
WITH u, m, actualRating, v,
     collect(r1.rating) AS ratings_u,
     collect(r2.rating) AS ratings_v,
     count(m_shared) AS commonMovies
WHERE commonMovies >= 3

WITH u, m, actualRating, v,
     gds.similarity.cosine(ratings_u, ratings_v) AS sim
WHERE sim IS NOT NULL AND sim > 0

MATCH (v)-[rv:RATED]->(m)
WITH u, m, actualRating, sim, rv.rating AS neighborRating
ORDER BY sim DESC
WITH u, m, actualRating,
     collect({sim: sim, rating: neighborRating})[0..10] AS neighs

WITH u, m, actualRating, neighs,
     reduce(num = 0.0, x IN neighs | num + x.sim * x.rating) AS weightedSum,
     reduce(den = 0.0, x IN neighs | den + abs(x.sim)) AS simSum
WHERE simSum > 0

RETURN
  u.id AS user,
  m.id AS movie,
  m.title AS title,
  actualRating,
  round(weightedSum / simSum, 3) AS predictedRating,
```

```
round(abs(actualRating - (weightedSum / simSum)), 3) AS absoluteError,
size(neighs) AS neighborsUsed;
```

4.3 Result

User	Movie	Title	Actual Rating	Predicted Rating	Absolute Error	Neighbors Used
123	100	Fargo (1996)	4.0	4.4	0.4	10
123	275	Sense and Sensibility (1995)	4.0	4.1	0.1	10

Table 15: User-based collaborative filtering prediction for a hidden rating.

4.4 Interpretation

The model predicts a rating of 4.4 for *Fargo (1996)*, while the true rating given by user 123 is 4.0, yielding an absolute error of 0.4. This result is relatively accurate and shows that the graph-based collaborative filtering approach is able to recover a realistic estimate from similar users. The prediction relies on 10 neighbors, which indicates that the estimate is supported by a sufficiently large set of comparable user profiles. Overall, this experiment suggests that similarity-based link prediction can be effectively used to infer missing user-movie ratings in the MovieLens graph.

5 Conclusion

This project explored the MovieLens dataset through a graph mining lens, progressively applying similarity measures, community detection, centrality analysis, and rating prediction using Neo4j and its GDS library.

The similarity analysis showed that users and movies tend to form tightly connected local structures. One notable finding is that behavioral similarity does not necessarily reflect demographic proximity: users with similar rating patterns can differ significantly in age or occupation, while users with similar watching habits tend to share stronger demographic alignment. This distinction matters in practice when choosing which similarity metric to use for recommendation.

Community detection with Louvain identified two dominant user groups with slightly different genre preferences, both centered around Drama. Centrality analysis added another layer: betweenness centrality revealed bridge users connecting distant parts of the graph, while PageRank highlighted the most influential users within each community's dense core.

For rating prediction, we used a user-based collaborative filtering approach and obtained reasonable results, with an absolute error of 0.4 on the tested example. It is worth noting that more advanced alternatives exist, such as model-based collaborative filtering or matrix factorization, which could better capture latent user and movie factors.

5.1 Limitations

The dataset being relatively small, perfect similarity scores appear frequently but are often driven by very low support and should be interpreted carefully. The Louvain algorithm produced only two meaningful communities, which suggests the similarity graph may be over-connected due to the 0.1 Jaccard threshold chosen. Finally, the link prediction evaluation was conducted on a single example, which is not sufficient to draw general conclusions.